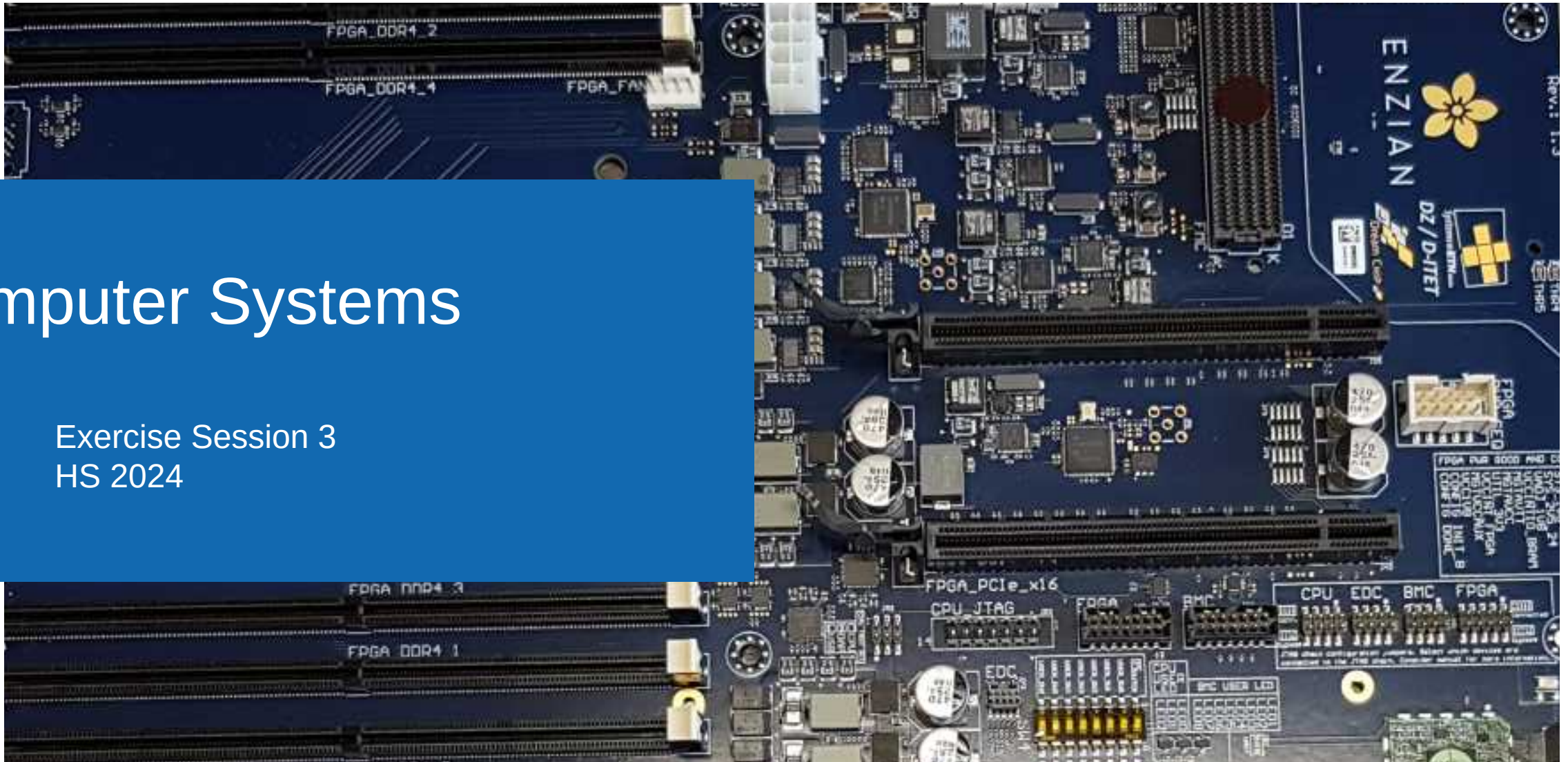


Computer Systems

Exercise Session 3
HS 2024



Program

1. Unix Comand Line – Questions ?
2. Recap Interprocess Communication
3. Recap IO
4. Discussion Assignment 1

How do processes communicate?

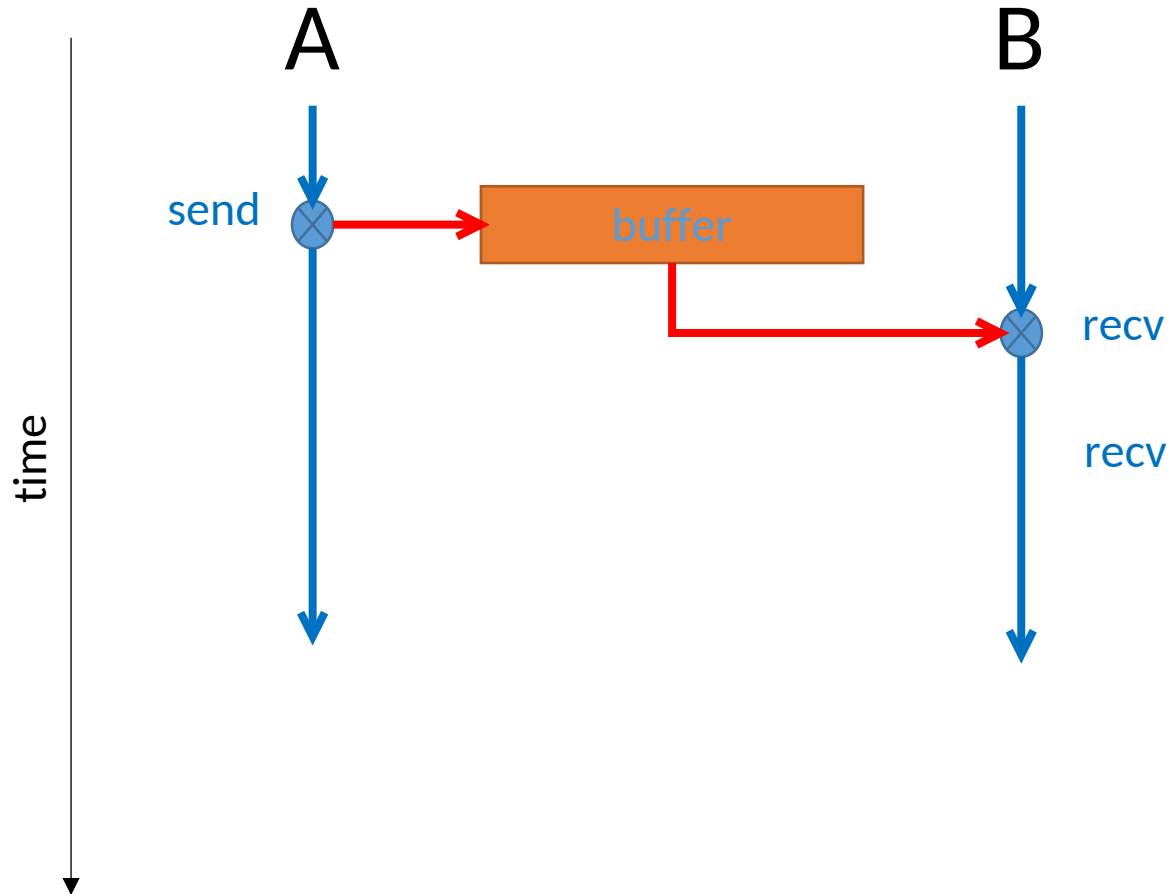
- Shared Memory

1. Semaphores/Locks
 1. Synchronization instructions (CAS, TAS, LL/SC)
2. Transactional Memory

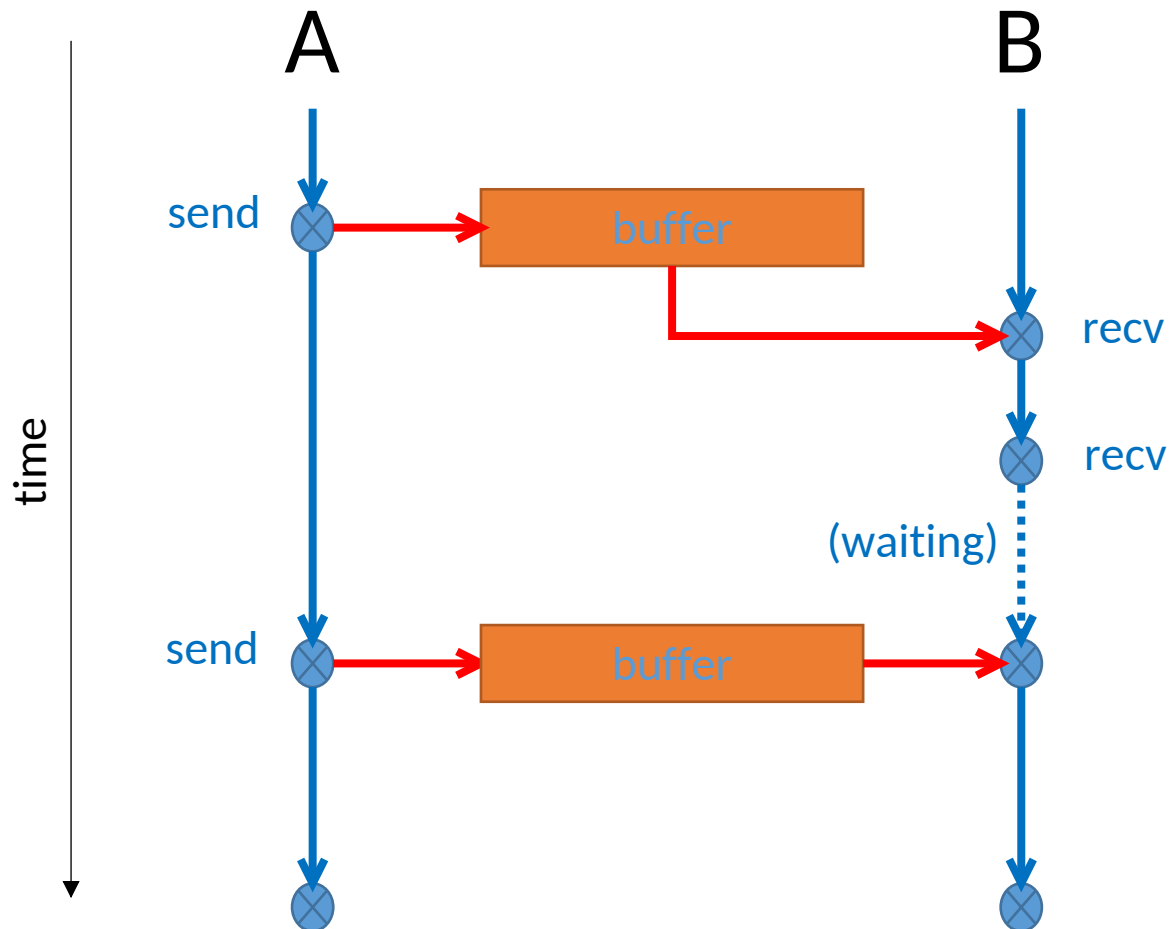
Message passing

1. Asynchronous/Synchronous
2. Blocking/non-blocking
3. Pipes
4. Upcalls/Signals
 1. SIGSEGV from memory management
 2. SIGKILL from other process
 3. Multiple signals
5. RPC

Channels: Asynchronous



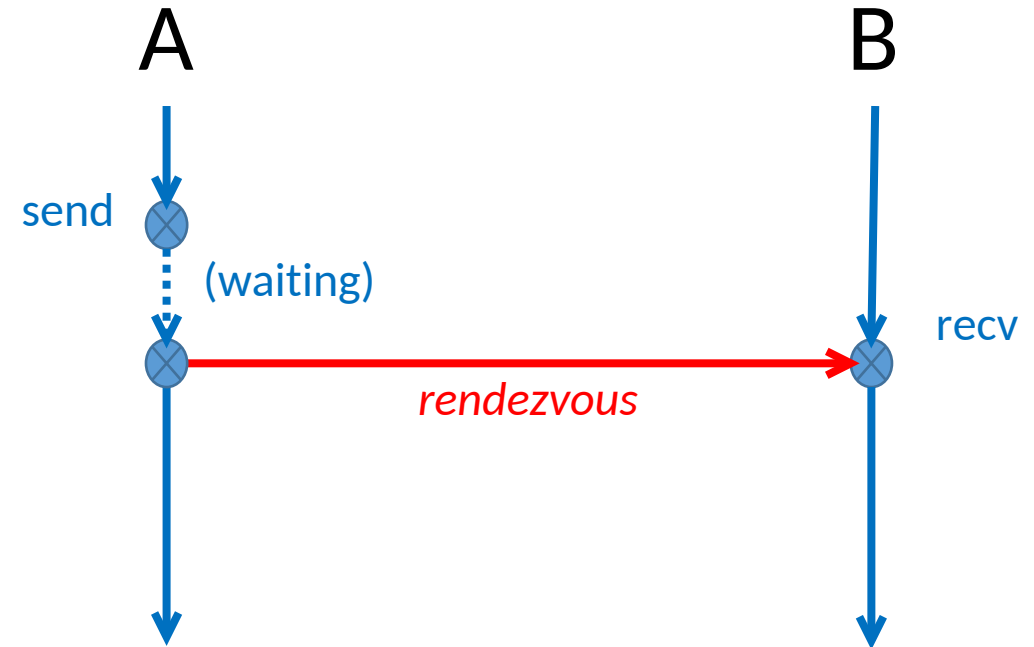
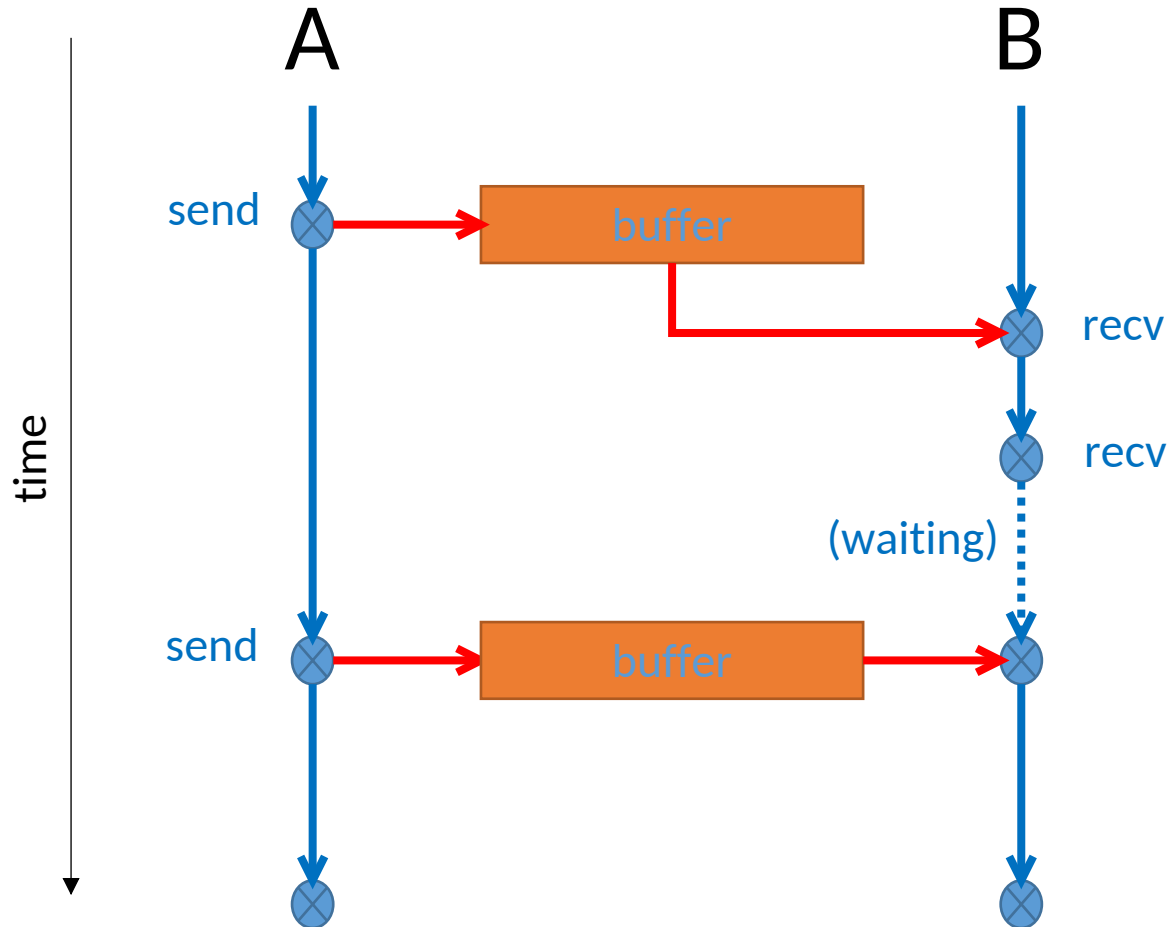
Channels: Asynchronous



Channels: Asynchronous

vs.

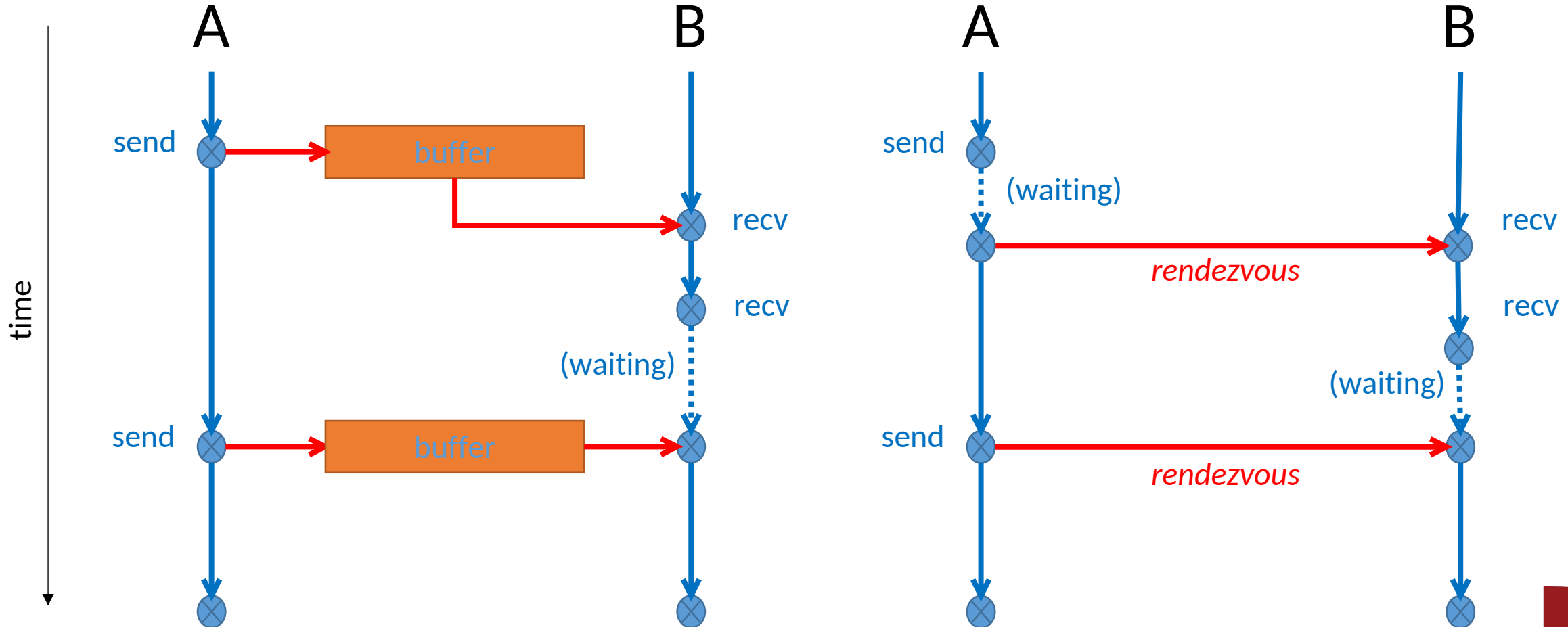
Synchronous



Channels: Asynchronous

vs.

Synchronous



Blocking / Non-blocking vs. Synchronous / Asynchronous

- If an operation cannot proceed:
 - Blocking ▲ block thread until it can.
 - Non-blocking ▲ immediately return «error: try again»
- Blocking: property of the **API**
- Synchrony (and buffering): property of the **channel**

Message Passing

Waiting for a text back like:



no synchronous mean of communication, but still waiting

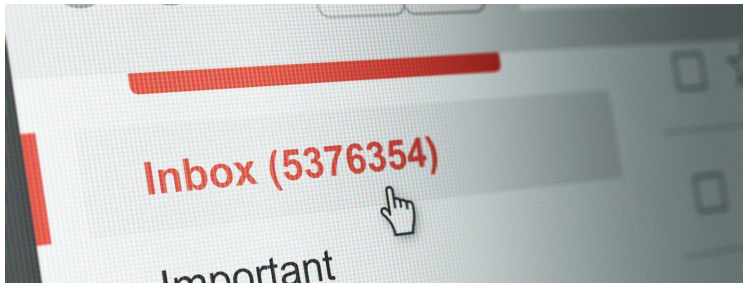
asynchronous

Blocking



call until someone picks up

synchronous



just work until you have time to react to message, communication succeeds even if no two parties act at the same time

Non blocking



call, if no one picks up then just continue working

Unix Pipes

- Basic (first) Unix IPC mechanism
- **Unidirectional**, **buffered** inter-process communication channel
- Creation:

```
int pipe(int pipefd[2])
```

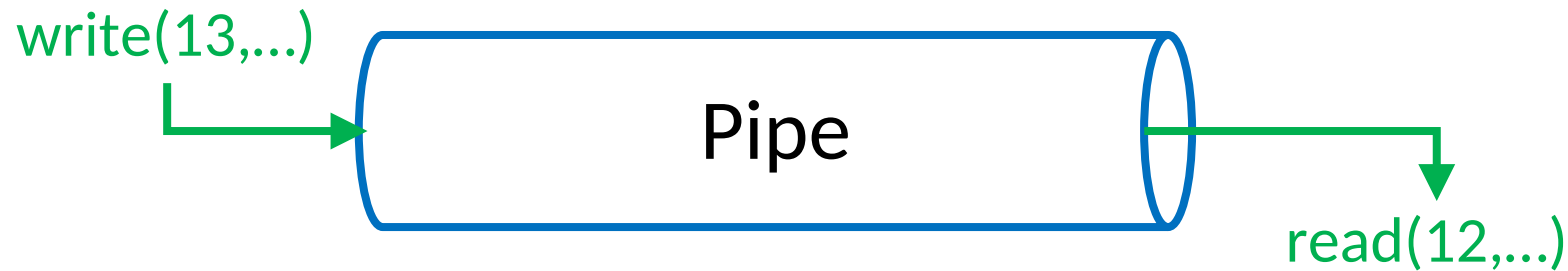
- Q. How to set up pipe between two processes?
- A. Don't! Create the pipe first, then fork...

Unix Pipes

```
int pipe(int pipefd[2])
```

returns

Index	End	Example
0	Reading	12
1	Writing	13



- Q. How to set up pipe between two processes?
- A. Don't! Create the pipe first, then fork...

Pipe idiom (man 2 pipe)

```
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <stdlib.h>
#include <sys/wait.h>

int main( int argc, char *argv[] )
{
    int pipefd[2];
    pid_t child_pid;
    char buf;

    if ( argc != 2 ) {
        fprintf(stderr, "Wrong number of arguments\n");
        exit( EXIT_FAILURE );
    }

    if ( pipe( pipefd ) == -1 ) {
        perror( "pipe" );
        exit( EXIT_FAILURE );
    }

    if ( (child_pid = fork()) < 0 ) {
        // Fork didn't work; give up
        perror( "fork" );
        exit( EXIT_FAILURE );
    }
    ...
```

Create pipe

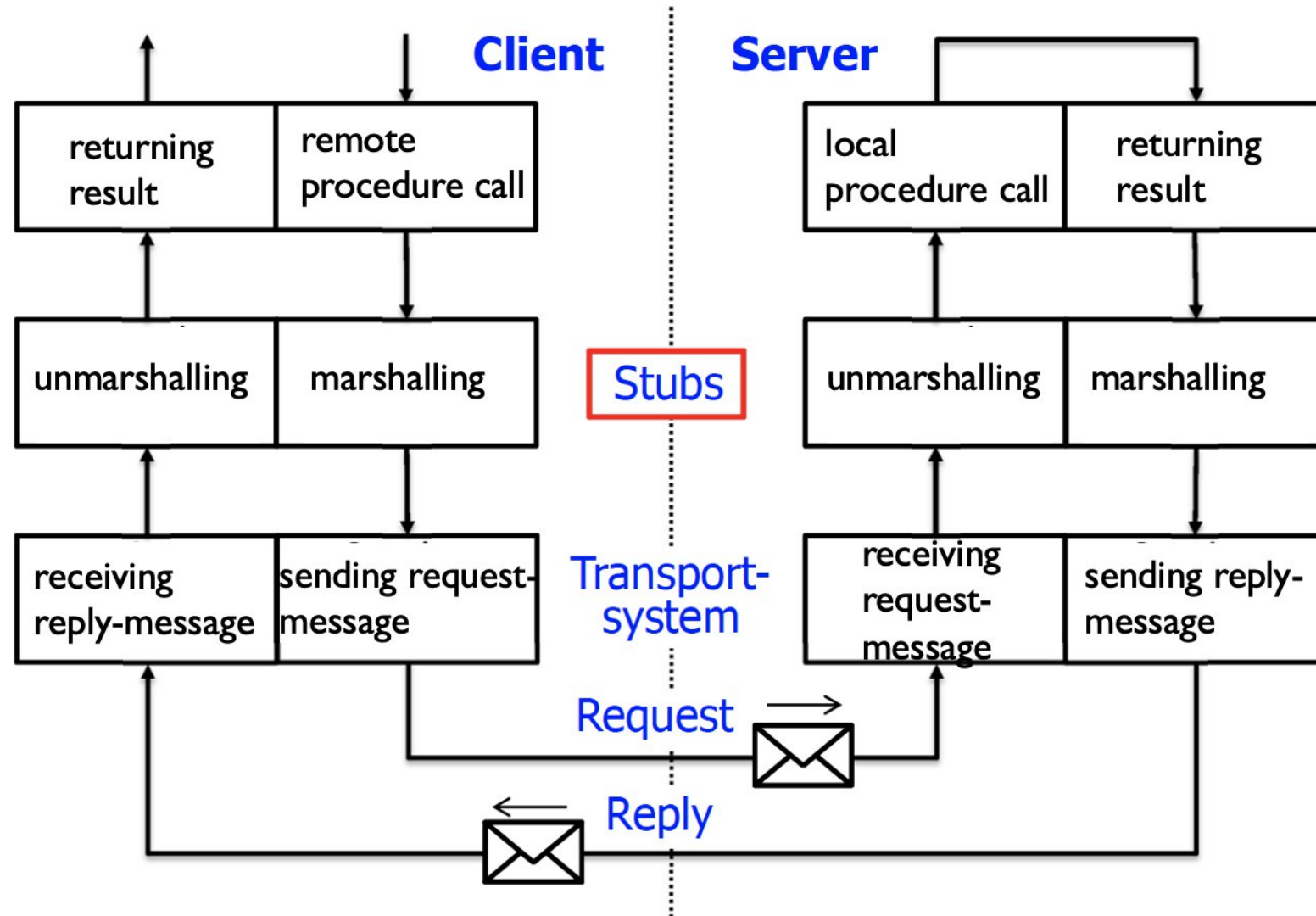
```
} else if ( child_pid == 0 ) {
    // We're in the child.
    // Close the write end of the pipe
    close( pipefd[1] );

    // Read from the read end
    while( read( pipefd[0], &buf, 1 ) > 0 ) {
        write( STDOUT_FILENO, &buf, 1 );
    }
    write( STDOUT_FILENO, "\n", 1 );
    close( pipefd[0] );
    exit( EXIT_SUCCESS );
} else {
    // We're in the parent
    // Close the read end of the pipe
    close( pipefd[0] );

    // Write the command line argument to the write end.
    write( pipefd[1], argv[1], strlen(argv[1]) );
    close( pipefd[1] );

    // Wait for the child to exit
    wait( NULL );
    exit( EXIT_SUCCESS );
}
```

Remote Procedure Call (RPC)



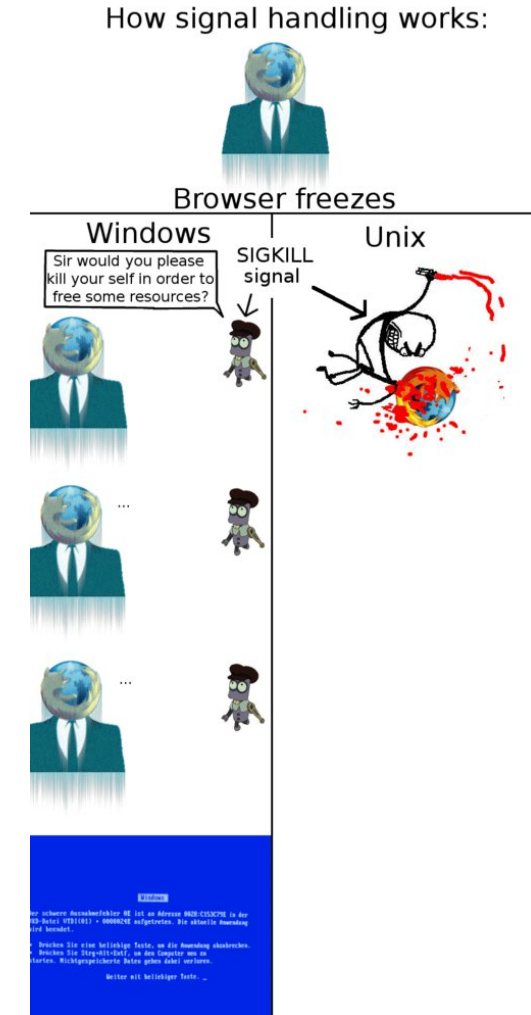
Signal types (some of them)

Name	Description / meaning	Default action
SIGHUP	Hangup / death of controlling process	Terminate process
SIGINT	Interrupt character typed (CTRL-C)	Terminate process
SIGQUIT	Quit character typed (CTRL-\)	Core dump
SIGKILL	kill -9 <process id>	Terminate process
SIGSEGV	Segfault (invalid memory reference)	Core dump
SIGPIPE	Write on pipe with no reader	Terminate process
SIGALRM	alarm() goes off	Terminate process
SIGCHLD	Child process stopped or terminated	Ignored
SIGSTOP	Stop process	Stop
SIGCONT	Continue process	Continue
SIGUSR1,2	User-defined signals	Terminate process

Etc. – see man 7 signal for the full list

Signal handlers

- Why do signal handlers need to be **reentrant** ?
- Better way to handle signals ?



What is a device?

Specifically, to an OS programmer:

1. Piece of hardware visible from software
2. Occupies some location on a **bus**
3. Set of **registers**
 1. Memory mapped or I/O space
4. Source of **interrupts**
5. May initiate **Direct Memory Access** transfers

Recap: what is a device driver?

- **Software** object (module, object, process, hunk of code) which abstracts a device
 - Sits between hardware and rest of OS
 - Understands device registers, DMA, interrupts
 - Presents **uniform interface** to rest of OS
- Device abstractions (“driver models”) vary...
 - Unix starts with “block” and “character” devices

Example: the xv6 console, output

- `kernel/printf.c:printf()` calls `console.c:consputc()`
- `console.c:consputc()` calls `uart.c:uartputc_sync()`
 - Spins until Transmit Holding Register is free
 - Then writes character and returns.
- As we saw, a bit basic.
 - No concurrency, interrupts, etc.

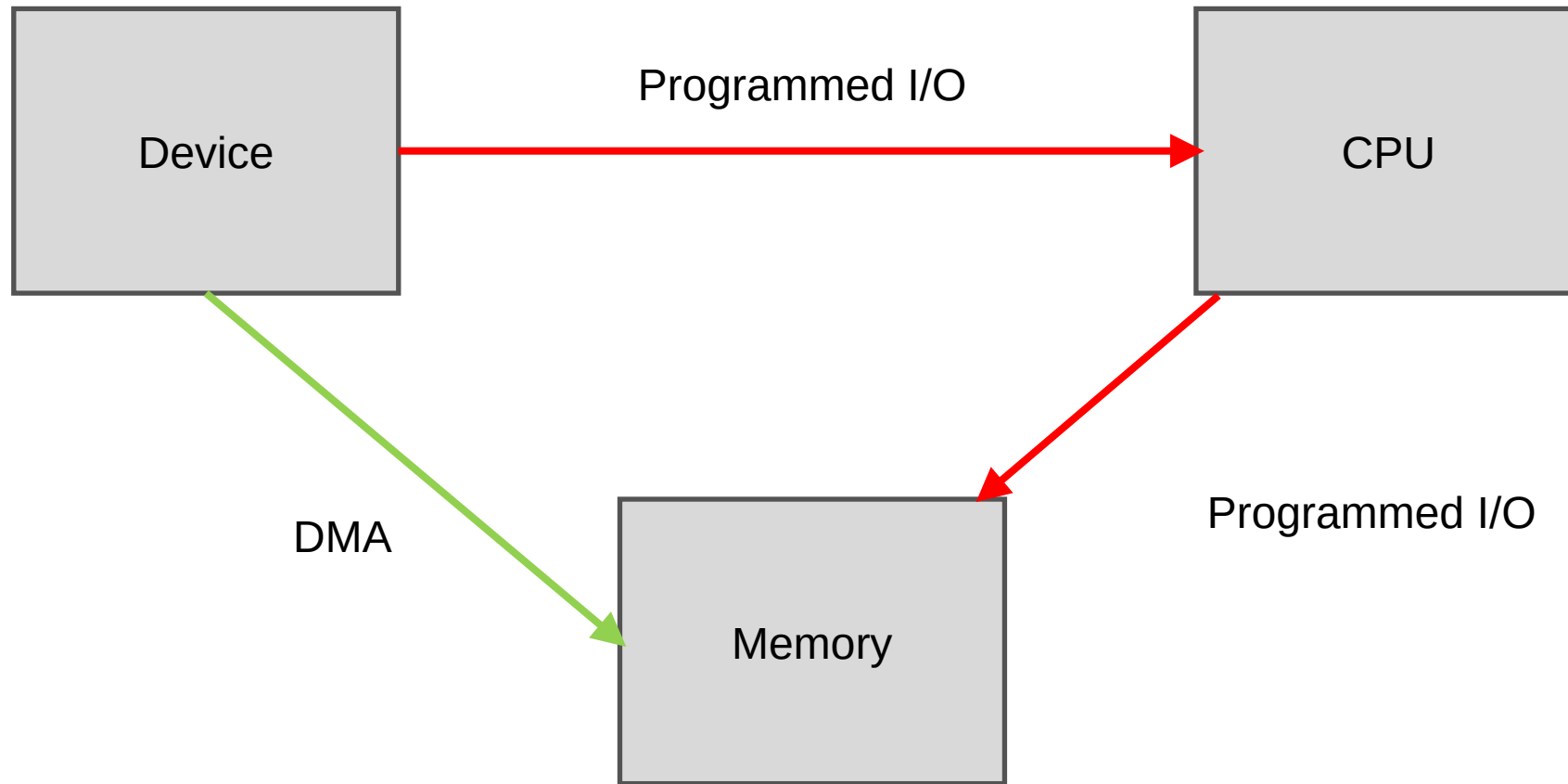
Example: the xv6 console, output

- Regular user writes come down to **uart.c:uartwrite()**
 - Takes a string as an argument
 - Waits until **tx_busy** is zero
 - Writes a character, sets tx_busy to zero, and loops.
- **uartintr()**:
 - Called when an interrupt is raised
 - Looks to see if the transmitter is ready
 - ▲ set **tx_busy** to zero and unblocks **uartwrite()**
 - Reads any received characters and calls **console.c:consoleintr()**

Example: the xv6 console, input

- Regular user reads come down to `console.c:consoleread()`
 - Tries to read a line from `cons.buf[]`, copying to user space
 - Sleeps if it has to.
 - Where does the buffer get filled? Well...
- `console.c:consoleintr(c)`:
 - Adds a character to the buffer, handling terminal controls
 - Called by `uart.c:uartintr()`...
- `uartintr()`:
 - Calls `uartgetc()` to get a character and calls `console.c:consoleintr()`
 - Repeats until no more received characters

Programmed I/O vs. Direct Memory Access



Evolution of device I/O

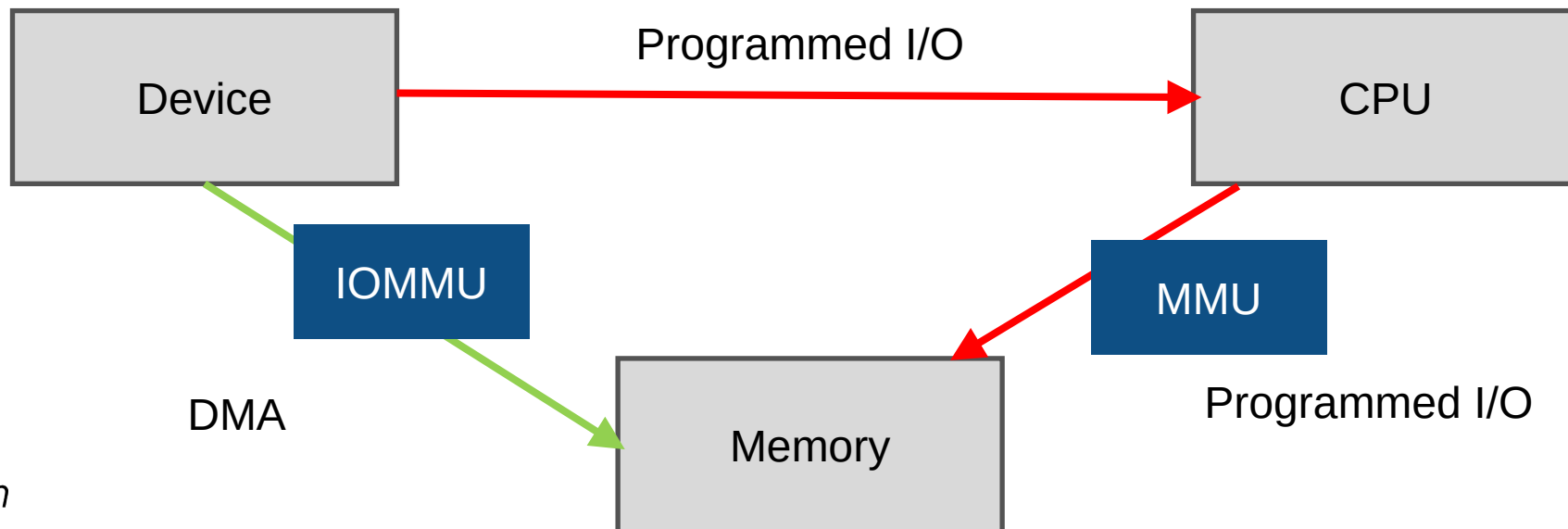
1. Programmed I/O with polling.
 1. Polling is too slow (CPU cycles, response latency)
 2. $\Rightarrow$ Interrupts notify CPU device needs attention
2. Programmed I/O with interrupts.
 1. CPU spends too much time copying data
 2. $\Rightarrow$ DMA allows CPU and device to operate in parallel
3. DMA

Direct Memory Access (DMA)

1. Avoid *programmed I/O* for lots of data (requires CPU)
2. Device needs to be able to do DMA (*DMA controller*)
 1. Generally built-in these days
3. Bypasses CPU to transfer data directly between I/O device and memory
 1. Doesn't take up CPU time
 2. Only one interrupt per transfer

I/O Protection

1. DMA operations can be dangerous to normal system operations because they directly access memory!
2. $\Rightarrow$ need IOMMU (I/O Memory Management Unit) to ensure that devices only access memory that they're supposed to access
3. Need to invalidate cache before transfer



Xv6 Lab 1 / Discussion - Preview

